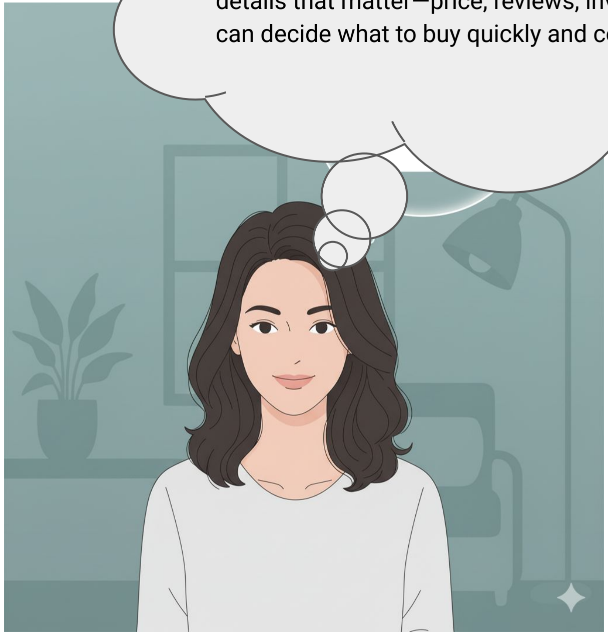
# Clothing Assistant Chatbot with Retrieval-Augmented Generation

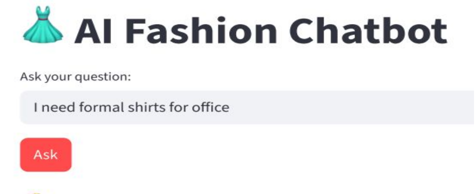

## GROUP 5

- Madhav (21f1004828@ds.study.iitm.ac.in)
- Manish (21f1005330@ds.study.iitm.ac.in)
- Neeraj (21f3002261@ds.study.iitm.ac.in)
- Harsha (23f2005056@ds.study.iitm.ac.in)
- Adarsh (21f2000283@ds.study.iitm.ac.in)

Maya realizes she has only one day left before the engagement party
She needs something elegant fast
Cocktail Chic
Maya tries browsing a major retailer for a navy blue jumpsuit.
300+ results even after filters.
Over 10,000 Results
She opens a silk-blend jumpsuit but starts worrying about shine, fit, and shipping cutoff.
She switches to the Help Center searching for shipping cutoff.
CUSTOMER SUPPORT PORTAL
She want to know about the Return policy on this product
Every question leads her away from the product page.
After 90 minutes, she's exhausted and still undecided.
She wishes buying one outfit wasn't such a chore.

Maya couldn't help thinking—why isn't shopping simple? Say goodbye to endless filters and guessing the perfect keywords. There should be a way to describe what you want just like you'd tell a friend and get a clear, reliable answer with accurate shopping data. Imagine asking a question and instantly getting an organized response with rich visuals and the details that matter—price, reviews, inventory—so you can decide what to buy quickly and confidently.



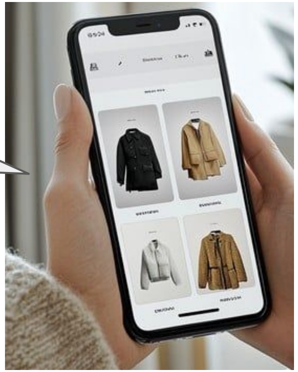


# Why a **Fashion Chatbot?**

- An **unified internal product** catalog ensures all recommendations are in stock and on-brand.
- **Query-based filtering** keeps the AI strictly aligned with the customer's exact request.
- **Context-aware fashion** rules provide relevant outfit suggestions.
- **Vector-based** Product catalog search.

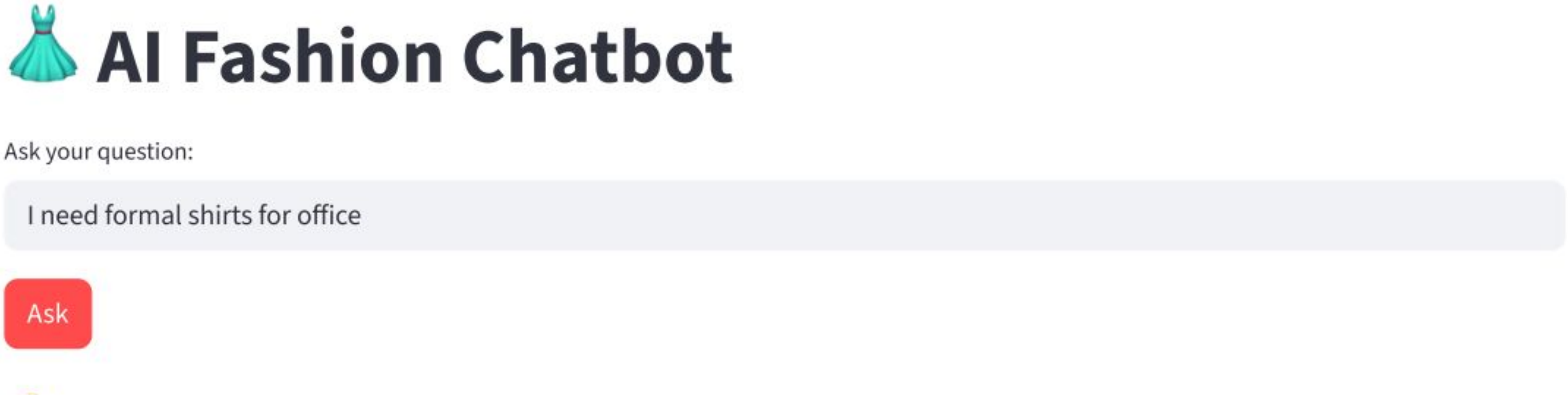
External Links
Forgot my context
Irrelevant matches

Internal Catalog
Contextual Memory
Smart Pairing



# Problem Statement

To build a **Clothing Assistant**. A chatbot designed to answer questions from store policy, previous purchases of customers, and suggest outfits based on query given and customer's previous purchases.



# Fashion Chatbot

## Store policy + Customer help desk

- What is the return policy?
- Can I get discount voucher if i return old clothes?
- What are the payment policies available?
- What is the refund conditions?
- How is the shipping done?

## AI stylist

- What about a red one?
- Help me find a blue shirt for my office
- Shirt that goes with cream pant
- Trendy tops in pastel colors for brunch
- Dress for winter

**User Query**

"What is the return policy?"

**Chat History**

- **Interaction History (last 5 entries):**
  - `10/25/25 10:01 AM`: Viewed "Black Linen Blazer"
  - `10/24/25 04:30 PM`: Added "Tailored Cotton Tee" to cart
  - `10/24/25 04:35 PM`: Abandoned Cart

**Previous Purchases**

**Previous Purchases (Past Transactions)**

- **Order ID `ORD7832` (Delivered 09/20/25):**
  - Item: White Tailored Shirt, Size M
  - Item: Beige Chino Pants, Size S
  - Status: Complete, No Return Filed

**Context**

| | |
|---|---|
| **Location** | New York, NY |
| **Preference: Style** | Minimalist, Casual Chic, Neutral Tones (Black, White, Beige) |
| **Preference: Fit** | Relaxed fit jeans, tailored tops |
| **Preference: Materials** | Cotton, Linen, Wool |

**Chat Response**

**Response Generation:**

1. *Initial Response (Standard):* "Our standard return policy allows for returns within **30 days** of delivery. Items must be unworn and have all original tags attached."

# Problems faced and solutions built

## Existing problems

- **Multi-dimensional similarity** — fashion items may match in one aspect (like color) yet differ in style, or pattern, making compatibility difficult for simple models to judge correctly
- **Agents lack personalization**
- **Agents do not stick to customer requests**

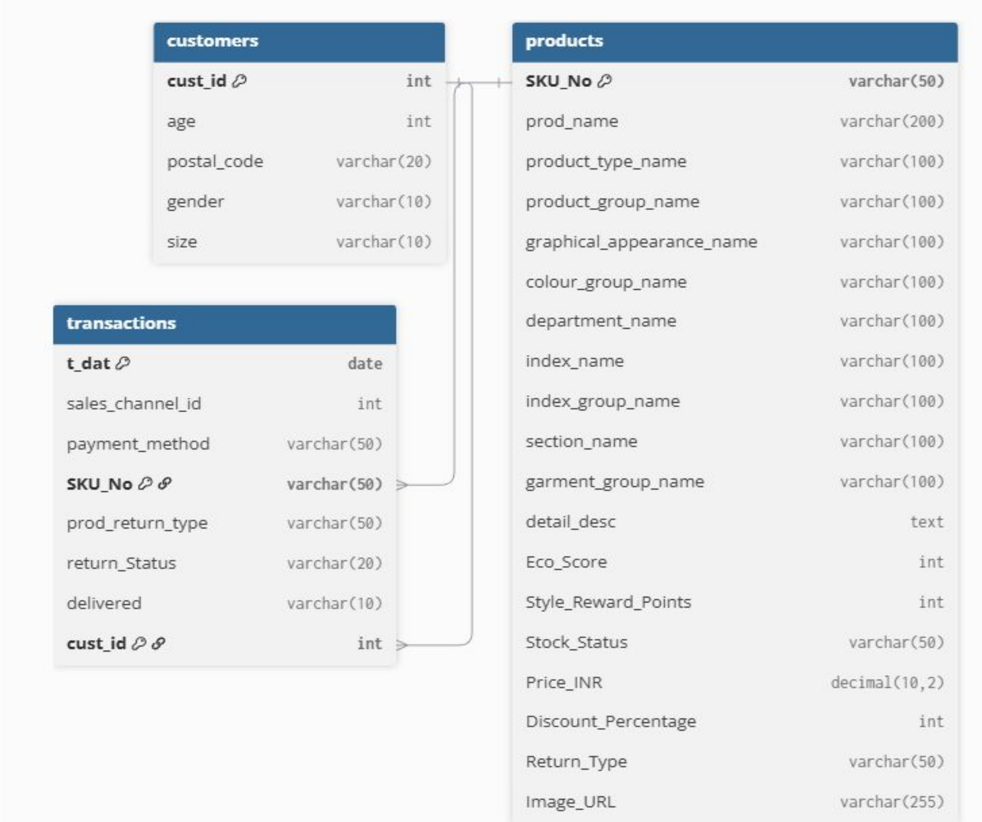
  Ex- when a customer asks for a pant for formal event, some ai models recommend shorts

- **No conversational memory** — agents forget previous context (e.g., occasion, colors, previous choices).

## Solutions

- **Personalization** — we take user history into account while recommending products
- **Stick to customer requests** —created another layer for this purpose, which gives very high weight to current user query to ensure that they are taken into consideration
- **Conversational History maintained.**
- **Ensures that all aspects are considered** – ensure that all the aspects of the products are considered, instead of focusing on only 1 aspect.

# Database scheme




The data is stored in a connected way so we can easily link every **customer**, **product**, and **transaction**.

This structure makes it easy to get complete details about any purchase, customer behaviour, or product performance.

# Policy **Dataset**

- A comprehensive policy framework was developed using ChatGPT, drawing references from established policy documents of leading clothing retailers such as H&M, Marks & Spencer (M&S), and Levi's.
- The formulated document encompasses key operational and customer-centric policies, including exchange, refund, and return procedures, garment recycling initiatives, loyalty and reward programs, as well as gifting policies.
- **Link to the policy Document:** Policy
- **Summary Stats on Policy Document:**
    - Minimum policy word count - 43
    - Max policy word count - 760
    - Median policy word count - 433
    - Mean/average policy word count - 396
    - Most common range for each policy- 350-500

# EDA on **Product Dataset**

- Dataset Overview:
  - **Total Records:** 105,542
  - **Total Features:** 25
- Data Cleaning
  - Filtered **out non-clothing categories** (Shoes, Bags, Underwear, Accessories, etc.)
  - Final dataset after filtering: **78,017 clothing items**
- Feature Types
  - **Numeric Columns:** 11
  - **Categorical Columns:** 15
- Missing Values
  - Only **detail_desc** column contains missing values (≈0.3%)
- Key Insights
  - **Most Common Product Type:** Trousers (11,169 entries)
  - **Most Frequent Color:** Black (22,670 entries)
  - **Most Common Appearance:** Solid pattern (49,747 entries)
  - **Most Frequent Item Name:** Dragonfly Dress (98 entries)

# Sample Product Dataset

| | SKU_No | prod_name | product_type_name | product_group_name | graphical_appearance_name | colour_group_name | department_name | index_name | index_group_name |
|---|---|---|---|---|---|---|---|---|---|
| 0 | 108775015 | Strap top | Vest top | Garment Upper body | Solid | Black | Jersey Basic | Ladieswear | Ladieswear |
| 1 | 108775044 | Strap top | Vest top | Garment Upper body | Solid | White | Jersey Basic | Ladieswear | Ladieswear |

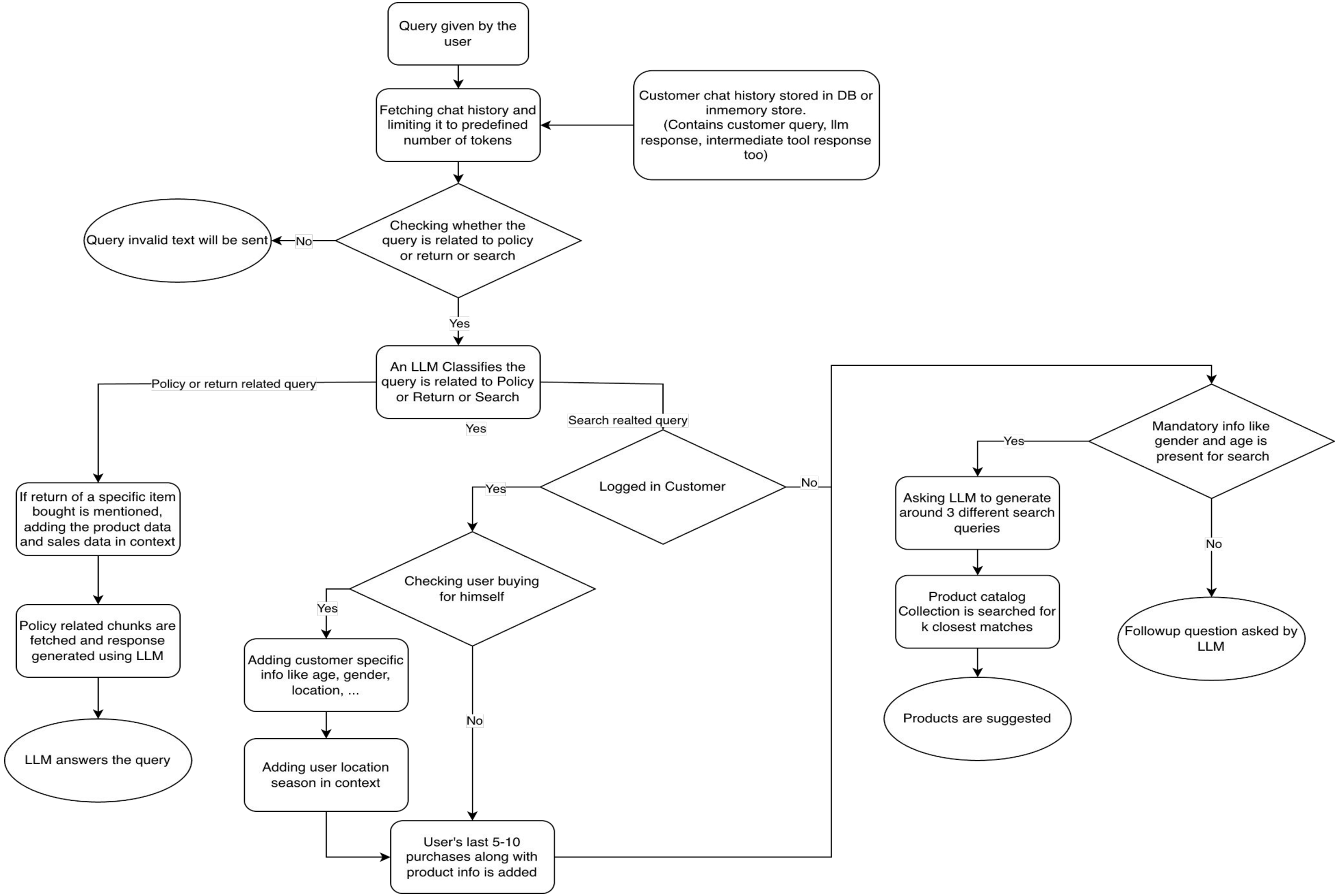
| section_name | garment_group_name | detail_desc | Eco_Score | Style_Reward_Points | Stock_Status | Price_INR | Discount_Percentage | Return_Type |
|---|---|---|---|---|---|---|---|---|
| Womens Everyday Basics | Jersey Basic | Jersey top with narrow shoulder straps. | 66 | 41 | Low Stock | 3862 | 32 | 7 days return |
| Womens Everyday Basics | Jersey Basic | Jersey top with narrow shoulder straps. | 82 | 73 | In Stock | 3959 | 11 | 15 days return |

# Embedders Used

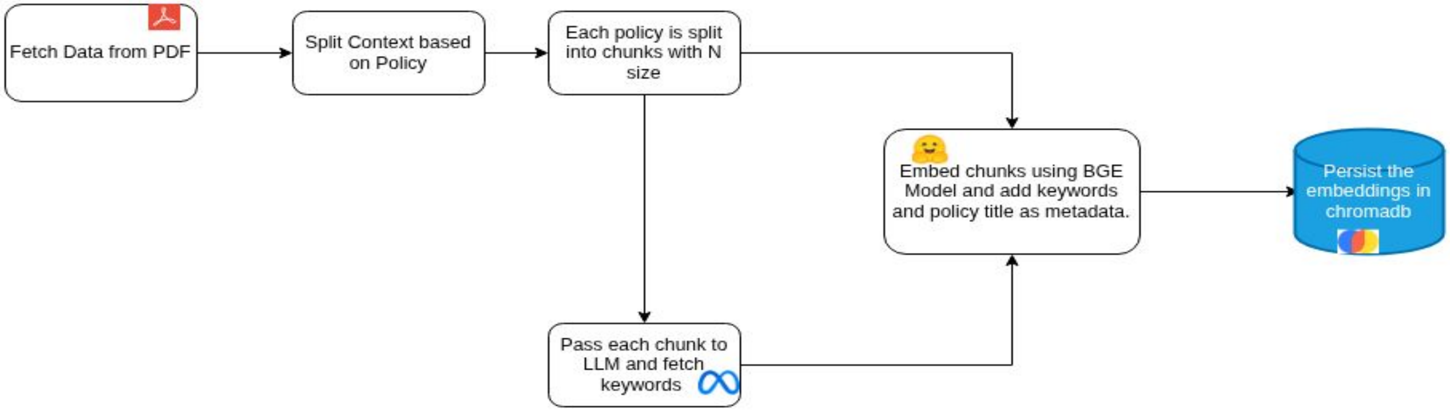
## BGE Sentence Transformer

- Specifically tuned for **dense retrieval in RAG**, where matching query and context meaning matters more than syntax.
- Delivers **strong performance in multilingual and domain-agnostic document search**, making it versatile for global or diverse datasets.
- Smaller models (like bge-small-en or bge-base-en) give **excellent performance-to-size ratio**, ideal for local or edge RAG setups.
- Strong alignment with LLM-based question answering pipelines — integrates well with vector databases (like FAISS, Milvus, Chroma).
- **Product RAG:** BAAI/bge-small-en-v1.5
- **Policy RAG:** BAAI/bge-base-en-v1.5

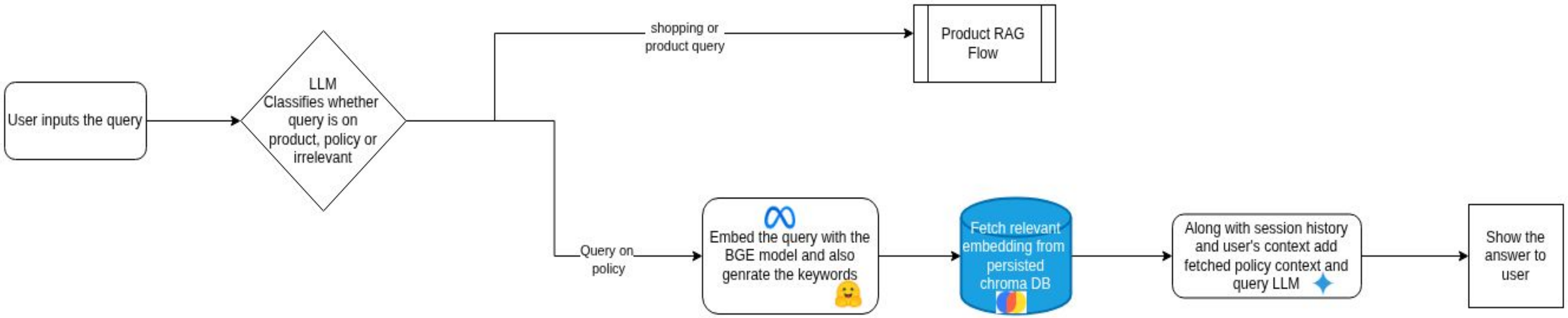
# Chatbot Flow



# Policy RAG



Fetch Data from PDF
Split Context based on Policy
Each policy is split into chunks with N size
Embed chunks using BGE Model and add keywords and policy title as metadata.
Persist the embeddings in chromadb
Pass each chunk to LLM and fetch keywords

User inputs the query
LLM Classifies whether query is on product, policy or irrelevant
shopping or product query
Product RAG Flow
Query on policy
Embed the query with the BGE model and also genrate the keywords
Fetch relevant embedding from persisted chroma DB
Along with session history and user's context add fetched policy context and query LLM
Show the answer to user

# **Evaluation** on Policy RAG

**RAGAS : R**etrieval-**A**ugmented **G**eneration **As**sessment

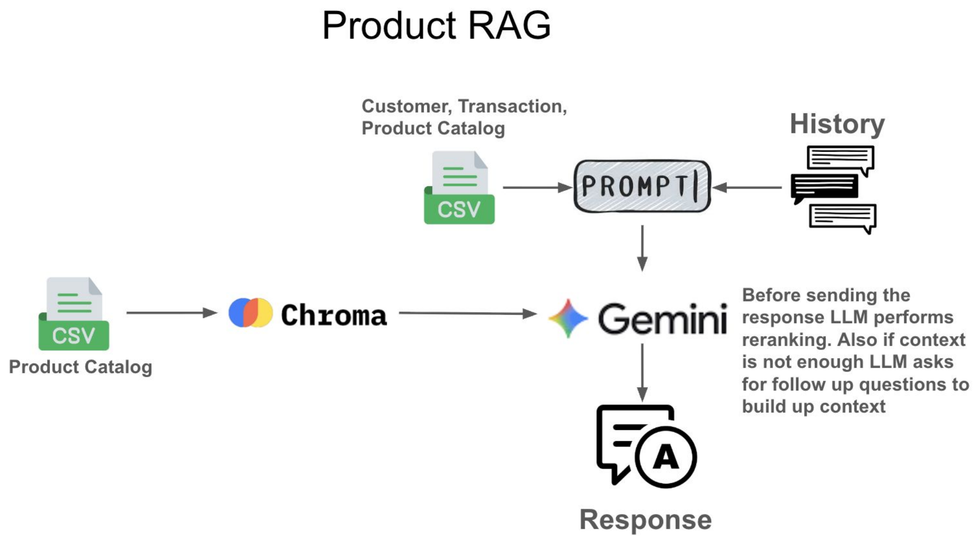
Based on the policy document, we formulated questions directly linked to the facts it outlined. Several questions drew from multiple policies, addressing areas such as the accrual and validity of loyalty points, as well as refund eligibility, including considerations related to GST and shipping charges.

Following Setup was used to perform Ragas evaluation:
**Question:** The user query provided as input to the RAG pipeline.
**LLM Answer:** The response generated by the RAG pipeline.
**Contexts:** The retrieved passages or documents from the external knowledge source that support the answer.
**Ground Truths:** The correct or reference answer used to assess the pipeline's performance.

Link to Policy Evaluation RAGAS: Evaluation

# Product RAG - Working

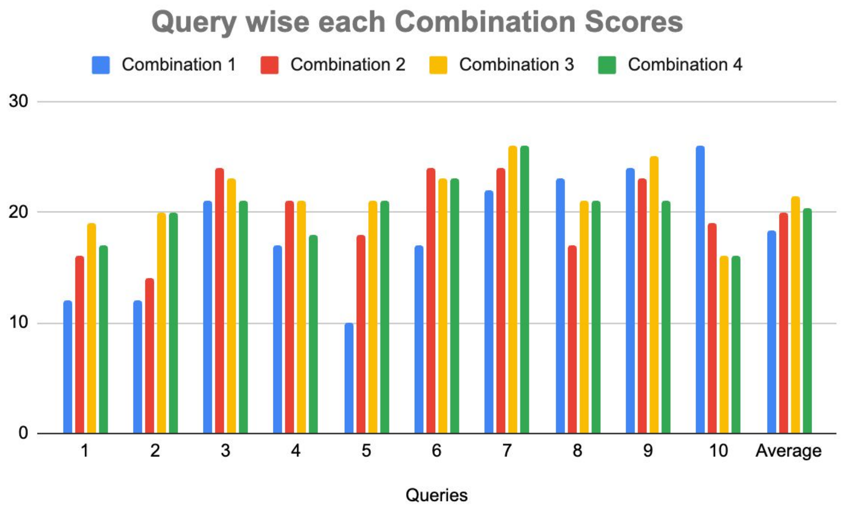
- Products stored in VDB
- Customer info, transactions stored in csv (can also be stored in RDB)
- User query along with customer context and chat history is used to create a prompt to search for relevant products in VDB
- The fetched products are reranked using LLM

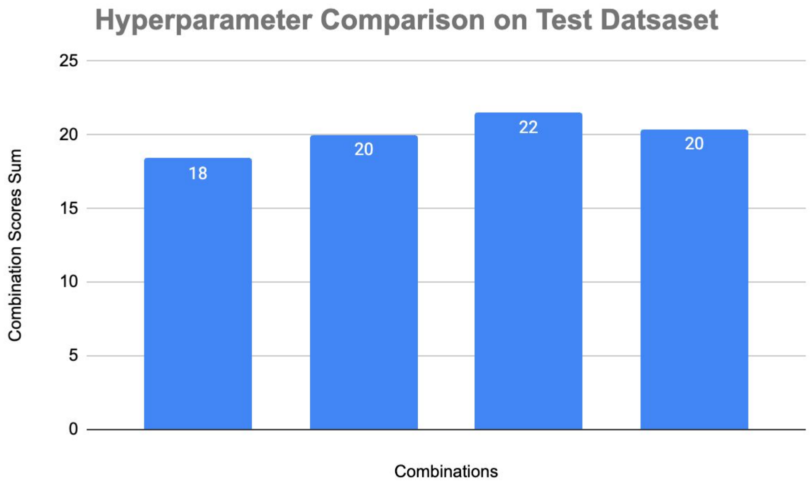


# Product RAG - Scoring

The scoring and hyperparameter tuning of product RAG is explained.

- Setup: Came up with 10 queries, along with chat history, customer context (age, gender, transaction history). The queries were run through the chat bot and the results were scored manually.
- Scoring sheet link: product_rag_scoring_sheet



# Chatbot Demo

## 👗 AI Fashion Chatbot

Ask your question:

What is the return policy of your store?

Ask

---

**AI Response:** At Clothing Retail, you may return or exchange items purchased within the specified return/exchange period mentioned on the product page.

**Return Process:**

1. Log in to your Clothing Retail account.
2. Go to "My Orders" and click on Return/Exchange for the respective item.
3. Choose Return or Exchange, specify the reason, and upload supporting images (if damaged).
4. Select your preferred refund mode (original payment method or Clothing Retail Wallet).
5. A pickup will be scheduled within 4–7 days or instructions for self-shipping will be provided.

**Eligibility:**

- If there's a size or fit issue after accepting the product, you can raise a return or exchange request through the standard process, subject to policy and stock availability.
- If a product is found wrong, damaged, or missing during Open Box Delivery, you will be eligible for a refund only, not replacement or exchange.

## 👗 AI Fashion Chatbot

Ask your question:

I need formal shirts for office

Ask

### 🛍️ Recommended Styles

Product

CA Gustavsberg TVP

Type: **Shirt**

Color: **White**

Price: **736 INR**

Discount: **20%**

Return Type: **15 days return**

SKU No: **395127001**

🛒 Add to Cart

Product

CA Gustavsberg TVP

Type: **Shirt**

Color: **Light Blue**

Price: **1805 INR**

Discount: **8%**

Return Type: **7 days return**

SKU No: **395127011**

🛒 Add to Cart

Product

TD check big Gingham/OPEN

Type: **Shirt**

Color: **Dark Blue**

Price: **580 INR**

Discount: **5%**

Return Type: **15 days return**

SKU No: **551910005**

🛒 Add to Cart

# Individual Contribution

- Defining problem statement - All members
- Literature review - All members
- Policy dataset and EDA - Madhav, Harsha
- Product dataset and EDA - Manish, Neeraj, Adarsh
- Policy Embedding - Madhav, Harsha
- Product Embedding - Adarsh, Neeraj
- Chat flow - Manish, Madhav
- Policy RAG hyperparameter tuning - Madhav, Manish
- Product RAG hyperparameter tuning - Manish, Harsha, Neeraj, Adharsh
- Coding - All members
- Policy and Product RAG testing - All members
- Documentation and PPT - All members
- Coordinators - Manish, Harsha

# Thank You